

REDIS - DATA SHARING / PUBSUB

2018001834 - Gabriel Belluco Perez

2021002000 - Melissa Aarantes Barbara da Silva

2022008537 - Pedro da Silva Furnaleta

SEMINÁRIO

Prof. Rafael Frinhani





Redis - DataSharing / PubSub

1 Introdução

Sistemas distribuídos modernos dependem de mecanismos eficientes de comunicação, compartilhamento de estado e coordenação entre processos executando de forma concorrente em diferentes máquinas. Nesse contexto, middlewares distribuídos tornam-se fundamentais para abstrair a complexidade da comunicação em rede e permitir integração desacoplada entre serviços.

Este trabalho apresenta o desenvolvimento de um protótipo distribuído utilizando Redis como middleware central de comunicação e compartilhamento de estado. A solução explora os paradigmas Publish/Subscribe (Pub/Sub) e Data Sharing, demonstrando conceitos relacionados à comunicação assíncrona, arquitetura orientada a eventos e desacoplamento entre componentes distribuídos.

A arquitetura proposta é composta por um cliente terminal e múltiplos workers especializados, responsáveis por diferentes operações do sistema. Toda a comunicação ocorre através do Redis, que atua simultaneamente como broker de mensagens e armazenamento compartilhado em memória.

O presente projeto está disponível no seguinte repositório do [Github](#).

2 Visão Geral do Redis

O Redis (*Remote Dictionary Server*) é um banco de dados multi-modelo em memória amplamente utilizado em arquiteturas distribuídas modernas devido à sua baixa latência e alta performance. Diferentemente de bancos tradicionais orientados a disco, o Redis opera principalmente em memória RAM, permitindo comunicação e acesso a dados em tempo real.

Além de funcionar como armazenamento chave-valor, o Redis também atua como cache, broker de mensagens, sistema de filas e middleware distribuído. Embora execute majoritariamente em memória, ele fornece mecanismos de persistência através de snapshots e backups em disco.

2.1 Estruturas de Dados do Redis

O Redis possui suporte nativo a múltiplas estruturas de dados, incluindo Strings, Hashes, Lists, Sets e Streams, permitindo modelagens eficientes para diferentes cenários distribuídos.

No protótipo desenvolvido, foram utilizadas principalmente estruturas HASH e LIST para armazenamento temporário de resultados e manutenção do histórico compartilhado do sistema.

2.2 Redis em Sistemas Distribuídos

Em sistemas distribuídos, o Redis é frequentemente utilizado como middleware centralizador responsável tanto pela comunicação quanto pelo compartilhamento de dados entre processos.

Seu uso permite baixo acoplamento entre serviços, comunicação assíncrona e compartilhamento eficiente de estado. Além disso, o Redis suporta escalabilidade horizontal através de clusters e mecanismos como *Sharded Pub/Sub*, utilizado para distribuição otimizada de mensagens.

2.3 Sistemas Distribuídos

Sistemas distribuídos consistem em múltiplos componentes independentes que cooperam através de comunicação em rede para executar tarefas coordenadas.

Esses sistemas buscam fornecer compartilhamento de recursos, escalabilidade, modularidade e paralelismo. Entretanto, sua complexidade aumenta devido à necessidade de sincronização, troca de mensagens e compartilhamento de estado entre processos distribuídos.

Nesse contexto, middlewares distribuídos tornam-se fundamentais para abstrair detalhes de comunicação e simplificar o desenvolvimento das aplicações.

2.4 Middleware de Comunicação

Middlewares distribuídos são camadas intermediárias responsáveis por facilitar a comunicação entre componentes distribuídos, abstraindo detalhes relacionados à rede e coordenação.

Entre os principais modelos utilizados destacam-se RPC, filas de mensagens, streaming de eventos e Publish/Subscribe.

No protótipo desenvolvido, o Redis desempenha exatamente esse papel intermediário, atuando simultaneamente como broker de mensagens e mecanismo de compartilhamento de estado.

2.5 Paradigma Publish/Subscribe

O paradigma Publish/Subscribe (Pub/Sub) é um modelo de comunicação assíncrona baseado na disseminação de eventos através de canais ou tópicos.

Nesse modelo, produtores publicam mensagens enquanto consumidores se inscrevem em canais específicos para receber notificações de interesse. Isso promove forte desacoplamento espacial e temporal, pois remetentes e destinatários não precisam se conhecer diretamente.

No Redis, o Pub/Sub é implementado através dos comandos:

```

1 PUBLISH
2 SUBSCRIBE
3 UNSUBSCRIBE

```

O Redis também suporta assinaturas por padrão e distribuição horizontal de mensagens através de *Sharded Pub/Sub*.

Entretanto, o modelo possui semântica *at-most-once*, ou seja, mensagens podem ser perdidas caso subscribers estejam desconectados no momento da publicação.

2.6 Data Sharing

O compartilhamento de dados (*Data Sharing*) é um dos principais objetivos de sistemas distribuídos, permitindo que múltiplos processos acessem e compartilhem informações de maneira coordenada.

No protótipo desenvolvido, o Redis atua como espaço compartilhado de dados distribuídos, permitindo que diferentes workers publiquem, consultem e atualizem informações compartilhadas em tempo real.

Essa abordagem reduz dependências diretas entre serviços e simplifica a coordenação do sistema.

2.7 Redis como Broker de Mensagens e Data Sharing

A arquitetura proposta utiliza o Redis em dois papéis principais: broker de mensagens Pub/Sub e armazenamento compartilhado de estado.

Como broker, o Redis realiza a distribuição assíncrona de eventos entre cliente e workers. Como mecanismo de Data Sharing, mantém histórico compartilhado, resultados temporários e sincronização indireta entre os serviços distribuídos.

Essa combinação permite que os componentes permaneçam desacoplados enquanto compartilham informações através de um espaço de dados comum.

3 Visão Geral da Arquitetura

A arquitetura desenvolvida utiliza Redis como middleware central responsável tanto pela comunicação distribuída quanto pelo compartilhamento de estado do sistema.

O sistema é composto por um Cliente CLI, Redis e três workers especializados: Calc Worker, Message Worker e File Worker. Todos os componentes executam de forma desacoplada e se comunicam exclusivamente através do Redis utilizando o paradigma Publish/Subscribe.

Esse modelo permite modularidade, concorrência e facilidade de expansão futura, possibilitando a adição de novos serviços sem alterações estruturais nos componentes existentes.

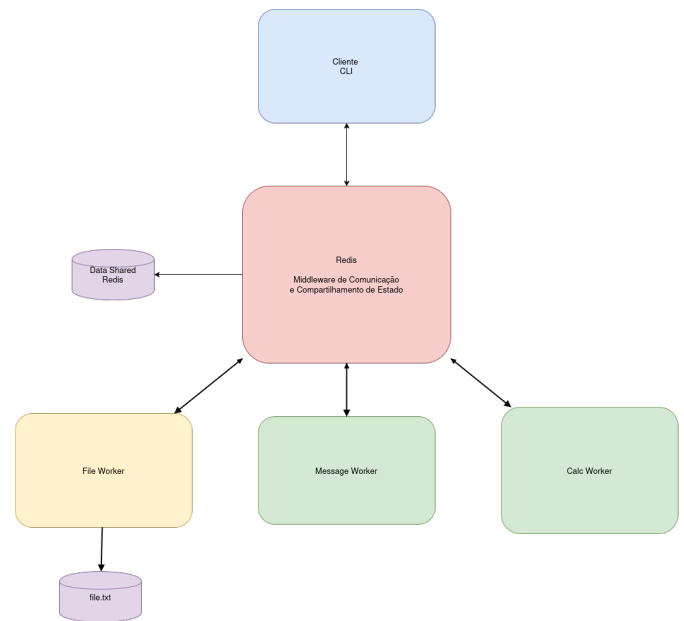


Figura 1: Arquitetura distribuída do sistema

A Figura 1 apresenta a organização dos principais componentes do sistema. O Redis atua como núcleo central da arquitetura, intermediando a comunicação entre cliente e workers através do modelo Pub/Sub e mantendo o estado compartilhado do sistema.

3.1 Cliente CLI

O Cliente CLI é responsável por iniciar as operações distribuídas do sistema. Ele gera identificadores UUID, serializa requisições em JSON, publica mensagens no Redis e monitora respostas assíncronas.

A publicação das mensagens ocorre através do método:

```

1 redis.publish(...)

```

As respostas são monitoradas utilizando:

```

1 pubsub.get_message(...)

```

Além disso, o cliente também realiza recuperação do estado compartilhado e controle de timeout das operações.

3.2 Workers Distribuídos

Os workers executam em containers independentes e permanecem inscritos no canal de requisições do Redis. Cada serviço filtra mensagens através do campo *operation*, processando apenas operações compatíveis com sua responsabilidade.

O Calc Worker é responsável pelas operações matemáticas de soma, subtração, multiplicação e divisão. Após o processamento, os resultados são armazenados no Redis e uma confirmação é publicada no canal de respostas.

O Message Worker processa mensagens textuais e mantém um histórico compartilhado das operações realizadas pelo sistema.

O File Worker realiza persistência em arquivo texto utilizando *append* síncrono. O serviço opera com apenas uma instância ativa para evitar concorrência simultânea durante operações de escrita.

3.3 Redis como Espaço Compartilhado

Além da comunicação Pub/Sub, o Redis também atua como espaço compartilhado de dados. Os resultados temporários são armazenados utilizando HASHES, enquanto o histórico compartilhado utiliza LISTS.

Essa abordagem permite persistência temporária, recuperação de estado e compartilhamento indireto de informações entre os serviços distribuídos.

4 Publicação de Requisições

O fluxo do sistema inicia quando o cliente gera um identificador UUID e serializa a requisição em JSON. Em seguida, a mensagem é publicada no canal `redis_requests`.

Todos os workers inscritos nesse canal recebem simultaneamente a mensagem publicada.

4.1 Processamento pelos Workers

Após receber a mensagem, cada worker verifica o campo `operation` para determinar se deve processar a requisição.

O serviço responsável executa a operação correspondente e produz o resultado da solicitação. Esse modelo permite concorrência entre os workers e reduz o acoplamento entre os componentes do sistema.

4.2 Compartilhamento dos Resultados

Os resultados processados são armazenados no Redis utilizando HASHES com chaves no formato:

```
1 resultado:<request_id>
```

Cada estrutura armazena informações como identificador da requisição, operação executada, status, resultado e timestamp de processamento.

4.3 Canal de Respostas

Após salvar os dados compartilhados, o worker publica uma notificação no canal `redis_responses`. O cliente monitora esse canal e recupera os dados utilizando `HGETALL`.

Dessa forma, o canal de respostas atua principalmente como mecanismo de sinalização assíncrona.

4.4 Estruturas Redis Utilizadas

O sistema utiliza os canais `redis_requests` e `redis_responses` para comunicação Pub/Sub.

Os resultados temporários são armazenados utilizando HASHES através do comando `HSET`, com TTL de 300 segundos para evitar crescimento indefinido do armazenamento.

O histórico compartilhado utiliza LISTS através dos comandos:

```
1 LPUSH + LTRIM
```

mantendo apenas as últimas dez operações realizadas no sistema.

5 Vantagens da Solução

A utilização do Redis Pub/Sub permitiu desenvolver uma arquitetura simples, modular e desacoplada. Entre as principais vantagens observadas destacam-se a comunicação assíncrona, o baixo acoplamento entre serviços, a facilidade de expansão horizontal e a baixa latência proporcionada pelo Redis operando em memória.

Além disso, o Redis também atuou como espaço compartilhado de estado, simplificando a coordenação entre os workers distribuídos.

5.1 Limitações do Redis Pub/Sub

O modelo Pub/Sub puro do Redis apresenta limitações importantes relacionadas à confiabilidade. Como o sistema utiliza semântica *at-most-once*, mensagens podem ser perdidas caso não existam subscribers ativos no momento da publicação.

Além disso, o Redis Pub/Sub não fornece persistência de mensagens, mecanismos de ACK, retry automático ou garantia de entrega. O Redis também atua como ponto único de falha da arquitetura proposta.

5.2 Trade-offs Arquiteturais

A arquitetura desenvolvida prioriza simplicidade operacional, baixo acoplamento e facilidade de implementação em detrimento de mecanismos avançados de confiabilidade.

Esse trade-off torna o sistema adequado para aplicações leves e comunicação em tempo real, porém menos apropriado para cenários críticos que exigem alta tolerância a falhas e garantia robusta de entrega.

5.3 Escalabilidade e Desacoplamento

O modelo adotado permite adicionar novos workers sem necessidade de alterações estruturais nos demais componentes do sistema.

Como toda a comunicação ocorre exclusivamente através do Redis, os serviços permanecem altamente desacoplados, favorecendo modularidade, paralelismo e escalabilidade horizontal da arquitetura.

6 Avaliação Experimental e Demonstração

A avaliação experimental teve como objetivo validar o funcionamento da arquitetura distribuída baseada em Redis, demonstrando a comunicação assíncrona entre processos distintos por meio dos paradigmas Pub/Sub e Data Sharing.

Os testes foram realizados em duas máquinas distintas conectadas à mesma rede local. Na Máquina A, foi executado o servidor Redis junto aos workers responsáveis pelo processamento das requisições. Na Máquina B, foi executado o cliente terminal, responsável por enviar as operações ao Redis e aguardar as respostas produzidas pelos workers.

6.1 Ambiente de Testes

O protótipo foi desenvolvido em Python, utilizando a biblioteca `redis-py` para comunicação com o Redis. A infraestrutura do lado servidor foi executada com Docker e Docker

Compose, permitindo isolar o Redis e os workers em serviços independentes.

Nessa arquitetura, o Redis atua como broker central da comunicação. O cliente publica requisições no canal `redis_requests`, enquanto os workers permanecem inscritos nesse canal aguardando mensagens. Após o processamento, os workers armazenam os resultados no Redis e publicam uma notificação de resposta no canal `redis_responses`.

6.2 Validação das Operações

Os testes realizados validaram o fluxo completo de comunicação entre cliente, Redis e workers distribuídos. Foram demonstradas três operações principais: envio e processamento de mensagens textuais, persistência de dados em arquivo texto no servidor e execução de operações matemáticas distribuídas.

A primeira operação validada foi o envio de uma mensagem textual pelo cliente. Nesse teste, o cliente publicou a requisição no canal `redis_requests`, o `message_worker` recebeu a mensagem, processou o conteúdo e disponibilizou a resposta ao cliente por meio do Redis.

```
==== Cliente Redis Pub/Sub ====
Redis: 192.168.0.14:6379
1 - Enviar mensagem de texto
2 - Alterar arquivo texto no servidor
3 - Realizar calculo
4 - Ver historico
0 - Sair
Escolha uma opcao: 1
Digite uma mensagem: Enviando mensagem de texto 1
Requisicao publicada em 'redis_requests' para 3 inscrito(s).
Aguardando resposta...
Resposta do servidor: Mensagem recebida pelo servidor: Enviando mensagem de texto 1
Resultado salvo no Redis em: resultado:7bb3cf9e-bf68-4db5-a479-9ea535d50cbc
```

Figura 2: Envio e processamento de mensagem textual pelo cliente.

A segunda operação validada foi a alteração de um arquivo texto no servidor. Nesse caso, o cliente enviou o conteúdo a ser persistido, e o `file_worker` processou a requisição, gravando os dados no arquivo `file.txt`. Essa operação demonstra o processamento remoto da requisição e a persistência do resultado no lado servidor.

```
data > file.txt
1 [2026-05-07 23:50:54.316786] alo
2 [2026-05-07 23:52:37.505593] 32
3 [2026-05-08 01:07:42.507753] Alterando arquivo 1
4 [2026-05-08 01:07:48.319393] alterando arquivo 2
5 [2026-05-08 01:07:55.180567] alterando arquivo 3
6
```

Figura 3: Arquivo texto alterado no servidor após requisições do cliente.

A terceira operação validada foi a execução de um cálculo matemático. O cliente informou os operandos e o operador desejado, publicando a requisição no Redis. Em seguida, o `calc_worker` processou a operação e retornou o resultado ao cliente, mantendo também o registro no histórico comparilhado.

```
C:\Users\flavi\Documents\Descubra\Pessoal\Redis\Redis-XRSC09> python Client/client.py
==== Cliente Redis Pub/Sub ====
Redis: 192.168.0.14:6379
1 - Enviar mensagem de texto
2 - Alterar arquivo texto no servidor
3 - Realizar calculo
4 - Ver historico
0 - Sair
Escolha uma opcao: 3
Digite o primeiro numero: 3213250
Digite o operador (+, -, *, /): /
Digite o segundo numero: 20
Requisicao publicada em 'redis_requests' para 3 inscrito(s).
Aguardando resposta...
Resposta do servidor: Resultado: 3213250.0 / 20.0 = 160662.5
Resultado salvo no Redis em: resultado:e760e7bf-c01c-4e9a-80c2-c9bfc88862d8
```

Figura 4: Execução de operação matemática distribuída.

Além dessas operações, também foi validada a consulta ao histórico de requisições, armazenado no Redis por meio do mecanismo de Data Sharing.

Durante a execução, cada requisição enviada pelo cliente recebeu um identificador único, permitindo associar a resposta publicada pelo worker ao pedido original. O resultado completo foi salvo em uma chave Redis no formato `resultado:<request_id>`, enquanto o histórico das operações foi mantido na lista `request_history`.

6.3 Evidências de Execução

A Figura 5 apresenta os logs do lado servidor, no qual é possível observar os workers conectados ao Redis e aguardando mensagens no canal `redis_requests`. Os logs também evidenciam o recebimento das requisições pelos workers responsáveis por cada tipo de operação.

```
C:\Users\flavi\Documents\Descubra\Pessoal\Redis\Redis-XRSC09> docker-compose up - build
Keep connections from any IP address on any network interface.
calc_worker-1 | calc_worker conectado em redis:6379.
message_worker-1 | message_worker conectado em redis:6379.
calc_worker-1 | Aguardando mensagens no canal 'redis_requests'...
message_worker-1 | Aguardando mensagens no canal 'redis_requests'...
file_worker-1 | Aguardando mensagens no canal 'redis_requests'...
message_worker-1 | message_worker recebeu: {'operation': 'message', 'content': 'Enviando mensagem de texto 1', 'request_id': 'cf55d55-b47a-4c2b-b01d-560b05b0a434'}
file_worker-1 | file_worker recebeu: {'operation': 'file', 'content': 'Alterando arquivo 1', 'request_id': 'c7788f89-cdf0-4a2e-8d61-560b05b0a434'}
file_worker-1 | file_worker recebeu: {'operation': 'file', 'content': 'Alterando arquivo 2', 'request_id': '4db3e409-ad0a-4080-b00a-7050e8a0a434'}
file_worker-1 | file_worker recebeu: {'operation': 'file', 'content': 'alterando arquivo 3', 'request_id': '5b2f2807-3b6f-4586-b091-5ca550b0a434'}
calc_worker-1 | calc_worker recebeu: {'operation': 'calc', 'a': 231321.0, 'b': 0.5, 'operator': '/', 'request_id': '38ced93-7413-4f08-970e-a473617e5'}
message_worker-1 | message_worker recebeu: {'operation': 'history', 'request_id': 'd0e2930e-5f42-41bf-a40b-fdc8222b9f99'}
```

Figura 5: Logs dos workers no servidor.

A Figura 6 apresenta a execução do cliente terminal. Nela, observa-se o envio de uma operação matemática, a publicação da requisição no canal `redis_requests`, o recebimento da resposta do servidor e a posterior consulta ao histórico armazenado no Redis.

```
C:\Users\flavi\Documents\Descubra\Pessoal\Redis\Redis-XRSC09> python Client/client.py
1 - Enviar mensagem de texto
2 - Alterar arquivo texto no servidor
3 - Realizar calculo
4 - Ver historico
0 - Sair
Escolha uma opcao: 3
Digite o primeiro numero: 433 + 142
valor invalido. Digite um numero.
Digite o primeiro numero: 231321
Digite o operador (+, -, *, /): *
Digite o segundo numero: 0.5
Requisicao publicada em 'redis_requests' para 3 inscrito(s).
Aguardando resposta...
Resposta do servidor: Resultado: 231321.0 * 0.5 = 115660.5
Resultado salvo no Redis em: resultado:38ced93-7413-4f08-970e-bd6a473617e5

==== Cliente Redis Pub/Sub ====
Redis: 192.168.0.14:6379
1 - Enviar mensagem de texto
2 - Alterar arquivo texto no servidor
3 - Realizar calculo
4 - Ver historico
0 - Sair
Escolha uma opcao: 4
Requisicao publicada em 'redis_requests' para 3 inscrito(s).
Aguardando resposta...
Resposta do servidor: [2026-05-08T01:09:30] calc (ok) - Resultado: 231321.0 * 0.5 = 115660.5
[2026-05-08T01:07:55] file (ok) - Arquivo file.txt atualizado com sucesso.
[2026-05-08T01:07:48] file (ok) - Arquivo file.txt atualizado com sucesso.
[2026-05-08T01:07:42] file (ok) - Arquivo file.txt atualizado com sucesso.
[2026-05-08T01:07:10] message (ok) - Mensagem recebida pelo servidor: Enviando mensagem de texto 1
Resultado salvo no Redis em: resultado:d0e2930e-5f42-41bf-a40b-fdc8222b9f99
```

Figura 6: Execução do cliente e consulta ao histórico.

Os resultados obtidos demonstram que a arquitetura implementada permite a comunicação desacoplada entre cliente e workers, utilizando Redis Pub/Sub para troca de mensagens e Redis Data Sharing para armazenamento e recuperação dos resultados processados.

7 Conclusão

O projeto desenvolvido demonstrou de forma prática a utilização do Redis como middleware distribuído utilizando os paradigmas Publish/Subscribe e Data Sharing.

A implementação permitiu validar conceitos fundamentais de sistemas distribuídos, incluindo comunicação assíncrona, desacoplamento entre serviços, compartilhamento de estado e arquiteturas orientadas a eventos.

A utilização do Redis como broker de mensagens e armazenamento compartilhado possibilitou a construção de uma arquitetura simples, modular e extensível, capaz de coordenar múltiplos workers distribuídos de maneira eficiente.

Além das vantagens observadas, o trabalho também evidenciou limitações importantes do modelo Pub/Sub puro, especialmente relacionadas à confiabilidade, persistência e garantia de entrega das mensagens.

Dessa forma, o protótipo atingiu os objetivos propostos ao demonstrar tanto o funcionamento prático quanto os trade-offs arquiteturais envolvidos na utilização do Redis em sistemas distribuídos modernos.

8 Divisão de Responsabilidades

1. **Membro 1 – Coordenador do Seminário e Integração (Melissa)** Responsável pelo planejamento e organização geral do seminário, incluindo distribuição de tarefas, acompanhamento das atividades, integração técnica do conteúdo produzido, comunicação com o docente e estruturação final da apresentação. Também atuou na revisão técnica dos slides e no apoio à preparação da demonstração do protótipo.
2. **Membro 2 – Analista da Tecnologia e do Paradigma (Pedro e Melissa)** Responsáveis pelo estudo do middleware e do paradigma de comunicação distribuída utilizado no seminário, incluindo elaboração do referencial conceitual, descrição da arquitetura da tecnologia, identificação de cenários de uso, vantagens, limitações e aplicações. Também colaboraram na modelagem da arquitetura do protótipo e na análise dos resultados obtidos.
3. **Membro 3 – Arquiteto do Protótipo (Pedro e Gabriel)** Responsáveis pela definição da arquitetura do sistema distribuído, identificação dos componentes da aplicação, modelagem do fluxo de comunicação entre processos, definição de bibliotecas e ferramentas, além da elaboração dos diagramas arquiteturais do sistema. Também apoiaram a implementação e documentação técnica do protótipo.
4. **Membro 4 – Desenvolvedor do Protótipo (Pedro e Gabriel)** Responsáveis pela implementação do protótipo demonstrativo, incluindo desenvolvimento das aplicações cliente e servidor, configuração do ambiente de

execução, integração de bibliotecas e garantia do funcionamento da comunicação distribuída. Também auxiliaram na realização de testes e preparação da demonstração prática.

5. **Membro 5 – Avaliação Experimental e Demonstração (Gabriel e Melissa)** Responsáveis conjuntamente pela execução dos testes do sistema distribuído, validação da comunicação entre processos, preparação do roteiro de demonstração, registro das evidências de execução e explicação do fluxo de comunicação durante a apresentação. Também contribuíram para a análise crítica da tecnologia e elaboração do material didático.

